

Final Exam Topics

12. Formal languages and automata

Note: This document was translated from Hungarian to English by AI. Please verify the information against the original Hungarian source before relying on it.

Formal languages and automata

Generative grammars and the Chomsky hierarchy. Normal forms of grammars. Regular expressions. Finite automaton. Pushdown automaton. Closure properties of language classes. Algorithmic problems of the regular and context-free language classes.

1 Formal grammars and the Chomsky hierarchy

1.1 Basic concepts

Alphabet, word

A finite nonempty set of symbols is called an alphabet.

Finite sequences formed from the elements of an alphabet V are called words or strings over V . The sequence of length 0 is called the empty word and is denoted by ε .

The set of words over the alphabet V (including the empty word) is denoted by V^* , the set of nonempty words by V^+ .

Concatenation

The word $x = uv$ is called the concatenation of the words $u, v \in V$.

Concatenation is associative, but (generally) not commutative.

V^* is closed under concatenation. That is:

$$u, v \in V^* \Rightarrow uv \in V^*$$

Furthermore ε can be regarded as an identity element with V^* and concatenation. That is:

$$u \in V^* \Rightarrow u\varepsilon \in V^*, \text{ and } \varepsilon u \in V^*$$

Other definitions

- $u, v \in V$. The word u is called a subword of v if $v = xuy$, ($x, y \in V$) holds. If, moreover, $xy \neq \varepsilon$, then u is a proper subword.
- $v = xuy$ ($x, y, u, v \in V$). Then:
 - If $x = \varepsilon$, then u is called a prefix of the word v
 - If $y = \varepsilon$, then u is called a suffix of the word v

- By the mirror image of a word $u \in V$ we mean the writing of its symbols in reverse order. (Notation: u^{-1})

1.2 Formal grammars

Language

If $L \subset V^*$, then L is regarded as a language over V .

The notation of the empty language (containing no word): $\emptyset$

A language L is a finite language if it contains a finite number of words, otherwise an infinite language.

Operations on languages:

- $L_1 \cup L_2 = \{u \mid u \in L_1 \text{ or } u \in L_2\}$: the union of the languages L_1 and L_2
- $L_1 \cap L_2 = \{u \mid u \in L_1 \text{ and } u \in L_2\}$: the intersection of the languages L_1 and L_2
- $L_1 - L_2 = \{u \mid u \in L_1 \text{ and } u \notin L_2\}$: the difference of the languages L_1 and L_2
- $\bar{L} = V^* - L$: the complement of $L \subseteq V^*$
- $L_1 L_2 = \{u_1 u_2 \mid u_1 \in L_1, u_2 \in L_2\}$: the concatenation of the languages L_1 and L_2
For every language: $\emptyset L = L \emptyset = \emptyset$ and $\{\varepsilon\} L = L \{\varepsilon\} = L$
- L^i : the i -th ($i \geq 1$) iteration of the language L (with respect to concatenation), and $L^0 = \{\varepsilon\}$
- $L^* = \bigcup_{i \geq 1} L^i$: the iterative closure of the language L

The operations union, concatenation, and iterative closure are called regular operations.

Grammar

Languages can be produced in many ways. One way of producing them with production systems is generating languages with a grammar.

By a generative grammar G we mean a quadruple (N, T, P, S) , where:

- N and T are disjoint alphabets, the alphabets of the nonterminal (N) and terminal (T) symbols.
- $S \in N$ is the start symbol.
- P is a set of ordered pairs (x, y) , where $x, y \in (N \cup T)^*$ and x contains at least one nonterminal symbol.
The elements of the set P are called rewriting rules.

Instead of the notation (x, y) we use the notation $x \rightarrow y$. (If $\rightarrow \notin (N \cup T)$)

Derivation

$G = (N, T, P, S)$, and $u, v \in (N \cup T)^*$. The word v is directly (in one step) derivable from the word u in G if

$$u = u_1 x u_2, v = u_1 y u_2 \text{ and } x \rightarrow y \in P \quad (u_1, u_2 \in (N \cup T)^*)$$

This is denoted $u \Rightarrow_G v$.

We say that the word v is derivable in k (≥ 1) steps from the word u in G if

$$\exists u_1, \dots, u_{k+1} \in (N \cup T)^* : u = u_1, v = u_{k+1} \text{ and } u_i \Rightarrow_G u_{i+1} \quad (1 \leq i \leq k)$$

The word v is derivable from the word u in G if $u = v$ or $\exists k \geq 1$ such that v is derivable from u in k steps.

Otherwise: The word v is derivable from the word u in G (notation: $u \Rightarrow_G^* v$) if $u = v$ or $\exists z \in (N \cup T)^*$ such that $u \Rightarrow_G^* z$ and $z \Rightarrow_G v$

Generated language

$L(G)$ is the language generated by the grammar $G = (N, T, P, S)$ if:

$$L(G) = \{w \mid S \Rightarrow_G^* w, w \in T^*\}$$

1.3 Chomsky hierarchy

A generative grammar $G = (N, T, P, S)$ is of type i ($i = 0, 1, 2, 3$) if the following hold for its rule set P :

Phrase-structure grammar (i=0)

No restriction

Context-sensitive grammar (i=1)

Every rule of P is of the form $u_1 A u_2 \rightarrow u_1 v u_2$, where $A \in N$ and $v \neq \varepsilon$, $(u_1, u_2, v \in (N \cup T)^*)$. Except for the rule $S \rightarrow \varepsilon$ (if it exists). Then S does not occur on the right-hand side of any rule.

Context-free grammar (i=2)

Every rule of P is of the form $A \rightarrow v$, where $A \in N, v \in (N \cup T)^*$.

Regular grammar (i=3)

Every rule of P is of the form $A \rightarrow vB$ or $A \rightarrow v$, where $A, B \in N, v \in T^*$.

A language L is of type i if it can be generated by a type- i grammar. The class of type- i languages is denoted by $\mathcal{L}_i$.

The type- i language classes have the following properties:

- $\mathcal{L}_3 \subset \mathcal{L}_2 \subset \mathcal{L}_1 \subset \mathcal{L}_0$
- Each of the language classes $\mathcal{L}_i$ ($i = 0, 1, 2, 3$) is closed under the regular operations.

2 Automata

The specification of formal languages is possible not only with generative but also with recognizing tools, that is, with the help of computational tools suitable for processing and identifying words. Such a tool is, for example, the automaton, which under the effect of a word, as input, can behave in two ways: either it accepts it, or it rejects it.

2.1 Finite automaton

Definition

The finite automaton is an ordered 5-tuple,

$$A = (Q, T, \delta, q_0, F)$$

where:

- Q - a finite nonempty set of states
- T - alphabet of input symbols
- $\delta : Q \times T \rightarrow Q$ - state transition function
- $q_0 \in Q$ - initial state
- $F \subseteq Q$ - set of accepting states

Operation:

The finite automaton operates through a sequence of steps performed in discrete time intervals. During each step the automaton reads the next input symbol and transitions into a state determined by the state transition function (based on the current state and input symbol).

Initially the finite automaton A is in the initial state q_0 and the reading head processes the first letter of the word $u \in T^*$ on the input tape. After this, the finite automaton, performing a sequence of steps, reads the input word u ; proceeding letter by letter it reads and gets into a new state.

After the finite automaton has read the last letter of the input word u , either $q \in F$, that is, it gets into an accepting state, and then the automaton accepts the word u , or the new state will not be an element of F , and then the automaton does not accept the word.

DFA - Deterministic finite automaton

The function δ is single-valued, so for every pair (q, a) , where $(q, a) \in Q \times T$, there exists a single state s for which $\delta(q, a) = s$ holds. Therefore this finite automaton is called deterministic.

NFA - Nondeterministic finite automaton

If we also allow a multi-valued state transition function, that is, $\delta : Q \times T \rightarrow 2^Q$, then we speak of a nondeterministic finite automaton. (In this case, as the current state, we can regard some element of a set of states rather than a single state.)

This means that the initial state can be replaced with an initial state set $Q_0 \subseteq Q$. (And it can also happen that for an input symbol a , $\delta(q, a)$ is empty for all the current states.)

Properties

The state transitions can also be written in the form of rules of the form

$$qa \rightarrow p$$

in the case $p \in \delta(q, a)$. Let us denote by M_δ the set of rules originating in the previous manner from the state transition function δ of the nondeterministic finite automaton $A = (Q, T, \delta, Q_0, F)$.

If for every pair (q, a) there is a single rule $qa \rightarrow p$ in M_δ , then the finite automaton is deterministic, otherwise nondeterministic.

- Direct reduction:

Let $A = (Q, T, \delta, q_0, F)$ be a finite automaton and let $u, v \in QT^*$ be words. We say that the automaton A reduces the word u to the word v in one step / directly. If there is a rule $qa \rightarrow p$ in M_δ , and there is a word $w \in T^*$ for which $u = qaw$ and $v = pw$ hold.

- Reduction:

The finite automaton $A = (Q, T, \delta, q_0, F)$ reduces the word $u \in QT^*$ to the word $v \in QT^*$ ($u \Longrightarrow_A^* v$) if $u = v$, or $\exists z \in QT^*$ for which $u \Longrightarrow_A^* z$ and $z \Longrightarrow_A v$ hold.

- The language accepted by the automaton:

By the language accepted/recognized by the finite automaton $A = (Q, T, \delta, q_0, F)$ we mean the set of words

$$L(A) = \{u \in T^* \mid q_0 u \Longrightarrow_A^* p, \quad q_0 \in Q_0 \text{ and } p \in F\}$$

. (The empty word is in the language $L(A)$ accepted by the automaton if and only if $Q_0 \cap F \neq \emptyset$).

- Theorem:

For every nondeterministic finite automaton A we can give a type-3 grammar G such that $L(G) = L(A)$ holds.

- Theorem:

For every type-3 grammar G we can give a finite automaton A such that $L(A) = L(G)$ holds.

After this the question arises: Is there a regular language that is recognizable by an NFA but not recognizable by a DFA?

Answer: There is not.

- Theorem:

For every NFA $A = (Q, T, \delta, Q_0, F)$ we can construct a DFA $A' = (Q', T, \delta', q'_0, F')$ such that $L(A) = L(A')$ holds.

2.2 Pushdown automaton

Definition

The pushdown automaton is an ordered 7-tuple

$$A = (Z, Q, T, \delta, z_0, q_0, F)$$

where

- Z - the finite set of stack symbols,
- Q - the finite set of states,
- T - the finite set of input symbols,
- $\delta : Z \times Q \times (T \cup \{\varepsilon\}) \rightarrow 2^{Z^* \times Q}$ - transition function
- $z_0 \in Z$ - the initial stack symbol,
- $q_0 \in Q$ - the initial state,
- $F \subseteq Q$ - the set of accepting states

By a configuration of the pushdown automaton we mean a word of the form uq , where $u \in Z^*$ is the current content of the stack and $q \in Q$ is the current state.

The initial configuration is z_0q_0 .

Operation:

Suppose that the reading head of the pushdown automaton A stands on the input symbol a , the pushdown automaton is in state q , and the symbol on the top of the stack is z . Let $\delta(z, q, a) = (u_1, r_1), \dots, (u_n, r_n)$, where $u_i \in Z^*$ and $r_i \in Q, 1 \leq i \leq n$. Then the next state of A will be some r_i and simultaneously it replaces z with the word u_i , and furthermore the reading head steps one cell to the right on the input tape.

If $\delta(z, q, \varepsilon)$ is not empty, then a so-called ε -transition can be performed.

If the input tape contains the word $w \in T^*$ and, starting from the initial configuration z_0q_0 and performing a sequence of steps, the pushdown automaton A reaches a configuration up , where p is an accepting state, then we say that A has accepted the word w .

Properties

- Direct reduction:

$$\alpha, \beta \in Z^*QT^*$$

The pushdown automaton A reduces the word α to the word β in one step ($\alpha \Rightarrow_A \beta$) if:

$$\exists z \in Z, p, q \in Q, a \in T \cup \{\varepsilon\}, r, u \in Z^* \text{ and } w \in T^*$$

such that:

$$(u, p) \in \delta(z, q, a) \text{ and } \alpha = rzqaw \text{ and } \beta = rupw$$

- Reduction:

The pushdown automaton A reduces the word α to the word β ($\alpha \Rightarrow_A^* \beta$) if either $\alpha = \beta$, or there is a finite sequence of words $\exists \gamma_1, \dots, \gamma_n \in Z^*QT^*$ such that $\alpha = \gamma_1, \beta = \gamma_n$, and

$$\gamma_i \Rightarrow_A \gamma_{i+1} \quad (i = 1, \dots, n-1)$$

- The language accepted by the pushdown automaton:

The language accepted (by accepting state) by the pushdown automaton A :

$$L(A) = \{w \in T^* \mid z_0q_0w \Rightarrow_A^* up, \text{ where } u \in Z^*, p \in F\}$$

- Determinism:

We can also give the mapping δ in the form of rules. We denote the rule set obtained this way by M_δ . So

1. $zqa \rightarrow up \in M_\delta$ if $(u, p) \in \delta(z, q, a)$
2. $zq \rightarrow up \in M_\delta$ if $(u, p) \in \delta(z, q, \varepsilon)$

We say that the pushdown automaton $A = (Z, Q, T, \delta, z_0, q_0, F)$ is deterministic if for every pair $(z, q) \in Z \times Q$:

1. $\forall a \in T : |\delta(z, q, a)| = 1$ and $\delta(z, q, \varepsilon) = \emptyset$
or
2. $|(z, q, \varepsilon)| = 1$ and $\forall a \in T : \delta(z, q, a) = \emptyset$

- Language accepted by empty stack:

The language $N(A)$ is accepted by the pushdown automaton A by empty stack if

$$N(A) = \{w \in T^* \mid z_0 q_0 w \xRightarrow{*}_A p, \text{ where } p \in Q\}$$

- Theorem:

For any context-free grammar G we can give a pushdown automaton A for which $L(A) = L(G)$ holds.

- Theorem:

For every pushdown automaton A we can give a context-free grammar G such that $L(G) = N(A)$ holds.

3 Properties and applications of regular languages

3.1 Normal form of type-3 grammars

Every type-3, that is, regular language can be generated by a grammar whose rules are:

- $X \rightarrow aY$, where $X, Y \in N$ and $a \in T$
- $X \rightarrow \varepsilon$, where $X \in N$

3.2 Regular expressions

Motivation

It is known that every finite language is regular. We also know that the language class $\mathcal{L}_3$ (the class of regular languages) is closed under the operations union, concatenation, and iterative closure.

Consequently, starting from a finite number of finite languages and applying the so-called regular operations listed above a finite number of times, we obtain a regular language.

The question is whether with this procedure we can produce every regular language, that is, whether this method is sufficient to describe the language class $\mathcal{L}_3$?

Definition

Let V and $V' = \{\varepsilon, \cdot, +, *, (,)\}$ be disjoint alphabets. We define the regular expressions over the alphabet V recursively as follows:

1. ε is a regular expression over V .
2. Every $a \in V$ is a regular expression over V .
3. If R is a regular expression over V , then $(R)^*$ is also a regular expression over V . [iterative closure]
4. If Q and R are regular expressions over V , then $(Q) \cdot (R)$ [concatenation] and $(Q) + (R)$ [union] are also regular expressions over V .

Remark: Every regular expression denotes (determines) some regular language. (E.g.: ε the language $\{\varepsilon\}$, $a + b$ the language $\{a\} \cup \{b\} = \{a, b\}$ and $a \cdot b$ the language $\{a\}\{b\} = \{ab\}$.) The regular expression is the syntax, how we interpret it is the semantics.

Axioms

P, Q, R are regular expressions. Then the following properties hold:

- Associativity:

$$P + (Q + R) = (P + Q) + R$$

$$P \cdot (Q \cdot R) = (P \cdot Q) \cdot R$$

- Commutativity:

$$P + Q = Q + P$$

- Distributivity:

$$P \cdot (Q + R) = P \cdot Q + P \cdot R$$

$$(P + Q) \cdot R = P \cdot R + Q \cdot R$$

- Identity element:

$$\varepsilon \cdot P = P \cdot \varepsilon = P$$

$$P^* = \varepsilon + P \cdot P^*$$

$$P^* = (\varepsilon + P)^*$$

The above axioms, however, are not yet in themselves sufficient to produce all regular expressions (with the help of substitution). We also need the following inference rule:

$$P = R + P \cdot Q \quad \wedge \quad \varepsilon \notin Q \quad \implies \quad P = R \cdot Q^*$$

Let us also add the symbol $\emptyset$ to the set of regular expressions, which denotes the empty language. (In this case we do not need the symbol ε , since $\emptyset^* = \{\varepsilon\}$). Thus, in the definition we can replace the symbol ε with the symbol $\emptyset$. Then we replace ε in the preceding axiom system with $(\emptyset)^*$ and consider one more axiom:

$$\emptyset \cdot P = P \cdot \emptyset = \emptyset$$

The above rules are sufficient to derive every valid equality between regular expressions.

Regular expressions and regular languages

Every regular expression denotes a regular (type-3) language, and conversely, for every regular language a regular expression denoting this language can be given.

(With this we have answered the question posed in the motivation.)

3.3 Linear grammars and languages

Definition

A context-free grammar $G = (N, T, P, S)$ is called linear if every rule of it is:

1. $A \rightarrow u$, $A \in N, u \in T^*$
2. $A \rightarrow u_1 B u_2$, $A, B \in N, u_1, u_2 \in T^*$

Furthermore, G is called left-linear, or right-linear, if $u_1 = \varepsilon$, or $u_2 = \varepsilon$, for every rule of form 2.

A language L is called linear, left-linear, or right-linear if there is a linear, left-linear, or right-linear grammar G for which $L = L(G)$ holds.

Linear and regular grammars, languages

The right-linear grammars are identical to the regular grammars (type-3).

Theorem:

Every left-linear grammar generates a regular (type-3) language.

4 Properties of context-free languages and their analysis**4.1 Normal forms of context-free grammars**

Normal forms of context-free grammars are grammars produced by a grammatical transformation that:

- satisfy certain syntactic conditions/properties
- are (generally) simpler in some respect than the original grammars
- generate the same language (so they belong to the same type)

Theorem:

For every context-free grammar $G = (N, T, P, S)$ we can construct an equivalent context-free grammar $G' = (N', T, P', S')$ such that: the right-hand side of every rule of G' is a nonempty word [except for the case when $\varepsilon \in L(G)$, in which case $S' \rightarrow \varepsilon$ is the only rule whose right-hand side is the empty word, and then S' does not occur on the right-hand side of any rule of G' .]

 ε -free grammar

The grammar G is ε -free if the right-hand side of none of its rules is the empty word.

Theorem:

For every context-free grammar G we can construct an ε -free context-free grammar G' for which $L(G') = L(G) - \{\varepsilon\}$ holds.

Chomsky normal form

We say that the context-free grammar $G = (N, T, P, S)$ is in Chomsky normal form if every single rule of it is:

- $X \rightarrow a$, where $X \in N, a \in T$
- $X \rightarrow YZ$, where $X, Y, Z \in N$

Theorem:

For every ε -free context-free grammar $G = (N, T, P, S)$ we can construct an equivalent context-free grammar $G' = (N', T, P', S)$ in Chomsky normal form.

Theorem (a consequence of the previous one):

For every context-free grammar G it is decidable whether a word u is in the language generated by the grammar G .

Reduced grammar

- A nonterminal of a context-free grammar is called inactive/non-active if no terminal word is derivable from it.
- A nonterminal of a context-free grammar is called unreachable if it does not occur in any word derivable from the start symbol.
- A nonterminal is called useless if it is either inactive, and/or unreachable.
- Whether a nonterminal A is reachable or active is decidable.
- A context-free grammar is reduced if every nonterminal of it is active and reachable.

Theorem:

For every context-free grammar we can construct an equivalent reduced context-free grammar.

4.2 Derivation tree

The derivations of context-free grammars can also be characterized by trees. The derivation tree gives information about the possibilities of producing a word. The derivation tree is a directed graph that satisfies special properties:

- The label of the root: S
- The label of the other vertices is some element of $(N \cup T)$

The derivation tree does not in every case give the order of the rules applied during the derivation. Two derivations are essentially the same if they differ only in the order of application of the rules.

Every derivation tree of a context-free grammar determines a unique (single) leftmost derivation. During the leftmost derivation, in every derivation step we have to replace the leftmost nonterminal.

Theorem:

For every context-free grammar it is decidable whether the language generated by it is the empty language or not.

4.3 Bar-Hillel Lemma

For every context-free language L we can give two natural numbers p and q such that every word in L that is longer than p is of the form

$$uxwyz$$

where $|xwy| \leq q$, $xy \neq \varepsilon$, and every word

$$ux^iwy^iv$$

is also in the language L for every integer $i \geq 0$ ($u, x, w, y, v \in T^*$).

Theorem:

It is decidable whether a context-free grammar generates an infinite language or not.